

1.1 Python: History, Features, Setup, Syntax, Variables, Data Types, Operators

- **History:**

Python was created by Guido van Rossum in 1991. It is an interpreted, high-level programming language. Its design philosophy emphasizes code readability with the use of significant indentation.

- **Key Features:**

1. Easy to learn and read.
2. Open-source.
3. Extensive standard library.
4. Portable and interactive.
5. Supports object-oriented, procedural, and functional programming.
6. High-level language with dynamic typing and automatic memory management.

- **Setting Up Path:**

After installing Python, the environment variable (PATH) must be set to the directory where Python is installed to execute Python programs from any directory.

- **Python Interpreter:**

The Python interpreter executes the Python code interactively. You can start it by typing `python` in the command line.

- **Basic Syntax:**

- **Comments:** `#` Single line comment
- **Indentation:** Indentation is used to define blocks of code instead of braces `{}`.
- **Variables:** No need to declare; variables are created when you assign a value.

```
x = 10
y = "Hello"
```

- **Data Types:**

- Numbers (int, float, complex)
- Strings (str)
- Lists
- Tuples
- Dictionaries

- Booleans (True, False)
- **Operators:**
 - **Arithmetic Operators:** +, -, *, /, %, ** (exponentiation)
 - **Comparison Operators:** ==, !=, >, <, >=, <=
 - **Logical Operators:** and, or, not
 - **Assignment Operators:** =, +=, -=, *=, etc.

1.2 Conditional Statements (If, If-Else, Nested If-Else)

- **If Statement:**
Executes a block of code if the condition is true.

```
if condition:  
    # code block
```

- **If-Else:**
Executes one block if the condition is true, otherwise another block.

```
if condition:  
    # block 1  
else:  
    # block 2
```

- **Nested If-Else:**
Multiple if statements inside one another.

```
if condition1:  
    if condition2:  
        # block  
else:  
    # another block
```

1.3 Looping (For, While, Nested Loops)

- **For Loop:**
Used to iterate over a sequence (list, tuple, string).

```
for item in iterable:  
    # block of code
```

- **While Loop:**

Repeats as long as the condition is true.

```
while condition:  
    # block of code
```

- **Nested Loops:**

A loop inside another loop.

```
for i in range(x):  
    for j in range(y):  
        # nested block
```

1.4 Control Statements (Break, Continue, Pass)

- **Break:**

Terminates the loop immediately.

```
for i in range(10):  
    if i == 5:  
        break
```

- **Continue:**

Skips the rest of the code inside the loop for the current iteration.

```
for i in range(10):  
    if i == 5:  
        continue
```

- **Pass:**

Used as a placeholder for future code. Does nothing.

```
if condition:
    pass
```

1.5 String Manipulation

- **Accessing Strings:**

Strings are indexed starting from 0.

```
python
Copy code
s = "Python"
print(s[0]) # Output: P
```

- **String Slicing:**

Extracting parts of a string.

```
s = "Python"
print(s[1:4]) # Output: yth
```

- **String Methods:**

- `len()`: Length of the string.
- `lower()`, `upper()`: Convert to lowercase/uppercase.
- `split()`: Split into a list.
- `replace()`: Replace a substring.

```
s = "Hello World"
print(s.upper()) # HELLO WORLD
```

1.6 Lists

- **Introduction:**

A list is a collection of ordered, changeable, and allows duplicate values.

- **Accessing Lists:**

List elements are accessed by index.

```
my_list = [1, 2, 3]
print(my_list[0]) # Output: 1
```

- **Basic Operations:**
 - Adding: `append()`, `insert()`.
 - Removing: `remove()`, `pop()`.
 - Slicing: `my_list[start:end]`.
- **Functions & Methods:**
 - `len()`: Length of list.
 - `sort()`: Sort the list.
 - `reverse()`: Reverse the list.

```
my_list = [3, 1, 2]
my_list.sort() # [1, 2, 3]
```

1.7 Tuples

- **Introduction:**

A tuple is an ordered and immutable collection.
- **Accessing Tuples:**

Accessed by index.

```
my_tuple = (1, 2, 3)
print(my_tuple[0]) # Output: 1
```

- **Basic Operations:**

Similar to lists but immutable.
- **Methods:**
 - `count()`: Returns the number of times an element appears.
 - `index()`: Returns the index of an element.

1.8 Dictionaries

- **Introduction:**

Unordered, changeable collection of key-value pairs.
- **Accessing Values:**

```
my_dict = {'name': 'John', 'age': 25}
print(my_dict['name']) # Output: John
```

- **Dictionary Methods:**

- `keys()`, `values()`, `items()`: Access dictionary content.
- `get()`: Get value for a key.

```
my_dict = {'name': 'John'}
print(my_dict.get('name')) # Output: John
```

1.9 Functions

- **Defining a Function:**

```
def function_name(parameters):
    # function body
```

- **Calling a Function:**

Functions are called by their name followed by parentheses.

- **Types of Functions:**

- **Built-in Functions:** Predefined in Python.
- **User-defined Functions:** Created by users.

- **Function Arguments:**

- **Positional Arguments:** Passed based on position.
- **Keyword Arguments:** Passed with parameter names.

- **Anonymous Functions (Lambda):**

Functions without a name.

```
add = lambda x, y: x + y
```

- **Global & Local Variables:**

- **Local:** Defined inside the function.
- **Global:** Defined outside and accessible throughout the program.

```
x = 10 # Global variable
def func():
    global x
```

```
x = 20 # Modify global variable
```

These notes cover the foundational aspects of Python programming and should help in preparing for exams.

40

You said:

can you a few question on this question for 1 or 2 marks

ChatGPT said:

ChatGPT

Here are a few 1 or 2 mark questions based on the notes:

1.1 Python History, Features, Syntax, Variables, Data Types, Operators

2. Who created Python and in which year?

- (1 Mark)

3. Mention two features of Python.

- (2 Marks)

4. How do you set up the Python environment variable PATH?

- (2 Marks)

5. What is the difference between = and == operators in Python?

- (1 Mark)

6. How are comments written in Python?

- (1 Mark)

1.2 Conditional Statements

7. What is the syntax of an if statement in Python?

- (2 Marks)

8. Write the syntax of an if-else statement.

- (2 Marks)

9. What is the purpose of a nested if-else statement?

- (1 Mark)

1.3 Looping

10. What is the main difference between a for loop and a while loop?

- (2 Marks)

11. Write a small code to print numbers from 1 to 5 using a for loop.

- (2 Marks)

12. When do you use a while loop?

- (1 Mark)

1.4 Control Statements

13. What is the purpose of the break statement?

- (1 Mark)

14. Explain the use of the continue statement.

- (1 Mark)

15. What does the pass statement do in Python?

- (1 Mark)

1.5 String Manipulation

16. How can you access the first character of a string `s = "Python"`?

- (1 Mark)

17. Write an example of string slicing to extract "yth" from the string "Python".

- (2 Marks)

18. What does the `upper()` method do in Python?

- (1 Mark)

1.6 Lists

19. What is a list in Python?

- (1 Mark)

20. How do you add an element to a list in Python?

- (2 Marks)

21. Write a small code to create a list and access the first element.

- (2 Marks)

1.7 Tuples

22. What is the main difference between a list and a tuple in Python?

- (1 Mark)

23. How do you create a tuple with three elements?

- (1 Mark)

1.8 Dictionaries

24. What is a dictionary in Python?

- (1 Mark)

25. How do you access a value in a dictionary using its key?

- (2 Marks)

26. Write a small code to create a dictionary with keys name and age.

- (2 Marks)

1.9 Functions

27. What is the syntax to define a function in Python?

- (2 Marks)

28. What is the difference between a local and global variable?

- (2 Marks)

29. What is a lambda function?

- (1 Mark)

30. Write an example of a function with no arguments and no return value.

- (2 Marks)